

VIDRIOS TEMPLEX

Sistema ERP — Manual Técnico

Versión 1.0 — Abril 2026



Confidencial — Uso interno

1. Descripción General del Sistema	4
1.1 Arquitectura General	4
1.2 Infraestructura de Producción	4
1.3 Variables de Entorno	4
Backend (backend-api/.env)	4
Frontend (frontend-web/.env)	4
2. Base de Datos	5
2.1 Modelos y Tablas	5
2.2 Migraciones de Esquema	6
2.3 Auditoría Automática	6
3. Entidad Central: ODP (Orden de Producción)	7
3.1 Estados de Producción	7
3.2 Estados de Facturación	7
3.3 Estados de Caja	7
3.4 Campos de Progreso (chk_*)	7
3.5 ODP Derivada (No Conformidad)	7
3.6 ODPItem — Campos Clave	7
4. Arquitectura Backend	8
4.1 Estructura de Archivos Clave	8
4.2 Patrón de Middlewares	8
4.3 Endpoints de la API (prefijo /api)	8
4.4 Convenciones de Código Backend	9
5. Flujos de Negocio Clave	10
5.1 Flujo SAP (Solicitud de Aprovisionamiento de Perfilaría)	10
5.2 Flujo Prospectos → ODP	10
5.3 Flujo PedidoPV (Vidrios Templados)	10
5.4 Flujo Salidas de Almacén	10
5.5 Flujo de Rutas de Instalación	10
5.6 No Conformidad	11
6. WebSockets (Socket.io)	12
6.1 Rooms	12
6.2 Emisión de Notificaciones	12
6.3 CORS de WebSocket	12
7. Autenticación y Control de Acceso (RBAC)	13
7.1 JWT	13
7.2 Roles	13

7.3 Campo Especial: puede_gestionar_pv	13
8. Arquitectura Frontend	14
8.1 Estructura de Features	14
8.2 Módulos y Rutas.....	14
8.3 Componentes Clave del Módulo ODP	14
8.4 Estado Global (Redux)	15
8.5 Componentes Printable	15
9. Módulo ROOT	16
9.1 Tabs del Panel ROOT	16
9.2 Health Checks	16
10. Aplicación Móvil	17
11. Guía de Desarrollo Local.....	18
11.1 Levantar el Backend	18
11.2 Levantar el Frontend.....	18
11.3 Levantar la App Móvil	18
11.4 Otros Comandos Útiles.....	18
11.5 Scripts One-Off (No ejecutar en producción)	18
12. Despliegue a Producción	19
12.1 Backend (Docker).....	19
12.2 Frontend (Netlify / Vercel)	19
12.3 CORS.....	19
12.4 Nginx (Reverse Proxy).....	19
13. Notas Técnicas Adicionales	20
13.1 Importación de Modelos	20
13.2 ODP sin Timestamps.....	20
13.3 Revertir Cambios (Auditoría)	20
13.4 Cron del PedidoPV	20
13.5 No Hay Tests Automatizados	20
13.6 Módulo de Cotizaciones	20

1. Descripción General del Sistema

Vidrios Templex ERP es una aplicación empresarial a medida que gestiona el ciclo completo de operaciones de la empresa: cotización → producción → instalación → facturación → cobro. Está organizada como monorepo con tres sub-proyectos independientes.

1.1 Arquitectura General

Sub-proyecto	Tecnologías	Puerto / Plataforma
backend-api	Node.js 20 + Express + TypeScript + Sequelize + Socket.io	3001
frontend-web	React 19 + MUI v5 + Redux Toolkit + Axios	3000 (dev)
mobile-app	Expo + React Native + Expo Router	Expo Go / APK

1.2 Infraestructura de Producción

- Base de datos: PostgreSQL en Supabase (AWS us-east-1), SSL obligatorio
- Almacenamiento de imágenes: Cloudinary, carpeta templex_instalaciones
- Frontend: Netlify / Vercel (CI=false para ignorar warnings de build)
- Backend: Docker multi-stage node:20-alpine, puerto 3001
- Reverse proxy: Nginx → /api/* al backend, SPA routing para el frontend

1.3 Variables de Entorno

Backend (backend-api/.env)

```
PORT=3001
JWT_SECRET=<secreto_jwt>
DATABASE_URL=postgresql://user:pass@host:5432/db
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
FRONTEND_URL=http://localhost:3000
```

Frontend (frontend-web/.env)

```
REACT_APP_API_URL=http://localhost:3001
CI=false
```

2. Base de Datos

El motor es PostgreSQL administrado en Supabase. Sequelize ORM se usa con `sync({ alter: false })` — solo crea tablas nuevas, nunca agrega columnas a tablas existentes. Toda migración de esquema requiere ALTER TABLE manual en el SQL editor de Supabase.

2.1 Modelos y Tablas

Modelo	Tabla	Descripción
ODP ☆	odp	Entidad central. Sin timestamps; usa fecha_creacion manual.
ODPItem	odp_items	Ítems de aluminio/vidrio de una ODP
Cliente	clientes	Clientes y empresas
Usuario	usuarios	Usuarios con rol RBAC. CHECK CONSTRAINT sobre el campo rol.
Cotizacion	cotizaciones	Cotizaciones pre-ODP
TomaMedidas	toma_medidas	Mediciones (ligada a ODP o Prospecto)
SAP	saps	Solicitud de Aprovisionamiento de Perfilaría
SAPItem	sap_items	Ítems de aluminio del SAP
OrdenCompra	ordenes_compra	ODC a proveedores (tipo: perfileria vidrio)
ODCItem	odc_items	Ítems de ODC, ligados a SAPItem u ODPItem
Pago	pagos	Registros de abonos y pagos
EvidenciaInstalacion	evidencias_instalacion	Fotos en Cloudinary
NoConformidad	no_conformidades	Reportes de defectos/no conformidades
NotaProduccion	notas_produccion	Notas internas de producción
HistorialEstadoODP	historial_estados_odp	Audit trail de cambios de estado
Vehiculo	vehiculos	Vehículos para instalación
RutaInstalacion	rutas_instalacion	Ruta diaria de instalaciones
RutaODP	ruta_odps	Join RutaInstalacion ↔ ODP
Prospecto	prospectos	Leads comerciales previos a ODP
CatalogoProducto	catalogo_productos	Catálogo de productos/perfiles
InventarioPerfileria	inventario_perfileria	Stock de perfiles de aluminio
MetaMensual	metas_mensuales	Metas mensuales por usuario
ConfiguracionGlobal	configuracion_global	Configuraciones del sistema
PedidoPV	pedido_pv	Pedidos de vidrios templados al proveedor PV
SalidaAlmacen	salidas_almacen	SA-XXXX por ODP facturada; una SA por ODP
AuditoriaLog	auditoria_log	Log global de INSERT/UPDATE/DELETE (25 modelos)
AlertasUmbral	alertas_umbral	Umbrales configurables para alertas ROOT

2.2 Migraciones de Esquema

IMPORTANTE: Sequelize sync({ alter: false }) NO agrega columnas a tablas existentes. Al agregar campos a modelos existentes se debe ejecutar manualmente:

```
-- Agregar columna nueva:  
ALTER TABLE nombre_tabla ADD COLUMN nueva_col VARCHAR(100);  
  
-- Agregar rol nuevo (tabla usuarios tiene CHECK CONSTRAINT):  
ALTER TYPE enum_usuarios_rol ADD VALUE 'nuevo_rol';  
-- Luego DROP y recrear el CHECK CONSTRAINT incluyendo el nuevo valor.
```

2.3 Auditoría Automática

Los hooks Sequelize en models/index.ts cubren 25 modelos. Cada INSERT/UPDATE/DELETE genera un registro en auditoria_log con:

- tabla, operacion, registro_id
- datos_anteriores / datos_nuevos (JSONB)
- usuario_id, ip_address, fecha

El contexto del usuario se inyecta mediante AsyncLocalStorage (requestContext.ts). El hook beforeUpdate guarda instance.previous() en instance._auditAntes antes del update; afterUpdate lo usa para grabar datos_anteriores.

3. Entidad Central: ODP (Orden de Producción)

La ODP es el núcleo del sistema. Tiene tres máquinas de estado independientes que avanzan de forma autónoma.

3.1 Estados de Producción

```
EN_ESPERA → VISITA_TECNICA → MEDICION → PEDIDO_PROVEEDOR
→ ALUMINIO_CORTADO → VIDRIO_RECIBIDO → ACCESORIOS_SEPARADOS
→ LISTO_INSTALAR → PROGRAMADA → INSTALADA → ENTREGADA
↳ PAUSADA (en cualquier punto)
```

3.2 Estados de Facturación

```
PENDIENTE → FACTURADA
```

3.3 Estados de Caja

```
PENDIENTE → ABONADO → CANCELADO
→ CREDITO_APROBADO
```

3.4 Campos de Progreso (chk_*)

La ODP tiene campos booleanos independientes del estado de producción:

- chk_medicion, chk_corte, chk_vidrio, chk_accesorios
- chk_ensamble, chk_matizado, chk_pelicula, chk_huacal, chk_carton

Se actualizan individualmente sin afectar el estado_produccion.

3.5 ODP Derivada (No Conformidad)

Al reportar una No_Conformidad, puede generarse una ODP hija con:

- odp_padre_id apuntando a la ODP original
- es_no_conformidad: true

3.6 ODPItem — Campos Clave

Campo	Valores / Descripción
color	Select con opciones fijas. Es el campo principal de color.
tipo_vidrio	Campo libre; se usa cuando color = "Otro"
prod	Tipo de proceso: CR, PV, LAM, etc.
estado_compra	pendiente en_odc en_existencia (tracking sin SAP)

4. Arquitectura Backend

4.1 Estructura de Archivos Clave

Archivo	Rol
app.ts	Configura Express, middlewares y rutas. Exporta app.
server.ts	Punto de entrada. Crea http.Server, adjunta Socket.io, arranca puerto. Exporta emitirNotificacion().
socketServer.ts	Lógica de rooms y eventos Socket.io
config/database.ts	Conexión Sequelize + SSL Supabase
config/upload.ts	Multer + Cloudinary storage
middlewares/authMiddleware.ts	Verifica JWT y agrega req.user
middlewares/rbacMiddleware.ts	Control de acceso por rol
utils/requestContext.ts	AsyncLocalStorage con userId, userName e ip por request
utils/notificaciones.ts	notificarCambioEstadoODP() — importar siempre desde aquí
models/index.ts	Asociaciones centralizadas. Siempre importar modelos desde aquí.

4.2 Patrón de Middlewares

Todas las rutas protegidas siguen este orden obligatorio:

```
authMiddleware → rbacMiddleware → controller
```

4.3 Endpoints de la API (prefijo /api)

Prefijo	Controlador	Descripción
/auth	auth.controller	Login y refresh de token JWT
/usuarios	usuario.controller	CRUD de usuarios
/clientes	cliente.controller	CRUD de clientes
/odp	odp.controller	CRUD de ODPs y cambios de estado
/produccion	produccion.controller	Control de producción y chk_*
/instalaciones	instalacion.controller	Gestión de instalaciones y programación
/evidencias	evidencia.controller	Subida/consulta de fotos en Cloudinary
/compras	odc.controller	SAPs, ODCs y vidrios sin SAP
/contabilidad	contabilidad.controller	Facturación y caja
/no-conformidad	no_conformidad.controller	Reportes de no conformidad
/configuracion	configuracion.controller	Parámetros globales del sistema
/notas-produccion	nota_produccion.controller	Notas internas

Prefijo	Controlador	Descripción
/catalogo	catalogo.controller	Catálogo de productos (también accesible para root)
/prospectos	prospecto.controller	Pipeline CRM y leads
/inventario-perfileria	inventario_perfileria.controller	Stock de perfiles
/rutas	rutas.controller	Rutas de instalación diarias
/dashboard	dashboard.controller	KPIs y gráficas resumen
/documentos	(PDF/Excel server-side)	Generación de documentos
/pedidos-pv	pedido_pv.controller	Pedidos de vidrios templados PV
/facturas-salidas	salidas_almacen.controller	Control facturas vs SA-XXXX
/root	root.controller	Panel exclusivo del rol root

4.4 Convenciones de Código Backend

- TypeScript estricto ("strict": true). No usar any explícito.
- Solo console.error y console.warn (no console.log).
- Validación de entrada con Zod en los controladores.
- Transacciones Sequelize para operaciones multi-tabla.

5. Flujos de Negocio Clave

5.1 Flujo SAP (Solicitud de Aprovisionamiento de Perfilaría)

El SAP es un documento intermedio entre la ODP y las compras de aluminio:

`ODP → SAP → SAPItem → OrdenCompra (ODC) → ODCItem`

- El controlador `sap.controller.ts` genera la lista de materiales de aluminio.
- Las ODCs de perfilaría tienen `sap_id` asociado.
- Las ODCs de vidrio tienen `sap_id=null` y usan `odc_items.odp_item_id`.

5.2 Flujo Prospectos → ODP

1. El comercial crea un Prospecto (lead).
2. Opcionalmente se registra una TomaMedidas al Prospecto (antes de ser ODP).
3. Al aprobarse el prospecto, se convierte en ODP. `Prospecto.odp_id` queda poblado.

5.3 Flujo PedidoPV (Vidrios Templados)

- Al crear una ODP con `proveedor_vidrio`, el backend auto-genera un PedidoPV (número consecutivo, base 6733 si no hay ninguno), fuera de la transacción principal.
- Alejandro (`puede_gestionar_pv=true`) asigna ítems de ODP al PedidoPV desde el tab "Por Gestionar".
- Si se seleccionan más de 12 ítems, se crean extensiones con sufijo -1, -2, etc.
- Ítems no asignados por Alejandro aparecen en el tab "Vidrios" de Compras para gestión directa (ODC sin SAP).
- ODPs sin `proveedor_vidrio` → todos sus ítems de vidrio van directo a Compras.

Estados del PedidoPV:

`PENDIENTE → ENVIADO → CONFIRMADO_PROVEEDOR → LLEGADO → VERIFICADO | PROBLEMA`

5.4 Flujo Salidas de Almacén

- `SalidaAlmacen` vincula una ODP facturada con su número SA (formato SA-XXXX, libre).
- Una sola SA por ODP (UNIQUE en `odp_id`).
- Al registrar SA, la ODP sale del tab "Facturadas" y pasa a "Con Salidas de Almacén".

5.5 Flujo de Rutas de Instalación

- `RutaInstalacion` agrupa múltiples ODPs vía `RutaODP` (tabla join).
- Tiene conductor, vehículo e instaladores (M:M vía tabla `ruta_instaladores`).
- ODPs con `forma_pago="credito"` se consideran pago OK para instalación automáticamente (sin requerir `CREDITO_APROBADO`).

5.6 No Conformidad

- Al reportar una No_Conformidad, puede generarse una ODP hija (odp_padre_id, es_no_conformidad=true).
- La ODP hija sigue el mismo flujo de producción que una ODP normal.

6. WebSockets (Socket.io)

El backend emite eventos en tiempo real. La configuración vive en server.ts y socketServer.ts.

6.1 Rooms

- Al conectarse, el cliente envía `join({ userId, rol })`.
- El servidor une al cliente a rooms `user_{id}` y `role_{rol}`.

6.2 Emisión de Notificaciones

Siempre usar el helper centralizado; nunca llamar `emitirNotificacion()` directamente desde los controladores:

```
import { notificarCambioEstadoODP } from '../utils/notificaciones';
```

- `notificarCambioEstadoODP()` notifica al asesor de la ODP + roles `jefe_produccion` y `compras` al cambiar el estado de producción.

6.3 CORS de WebSocket

Más restrictivo que el HTTP: solo `localhost:3000`, `FRONTEND_URL`, `*.netlify.app`, `*.vercel.app`.

7. Autenticación y Control de Acceso (RBAC)

7.1 JWT

El login genera un token JWT firmado con JWT_SECRET. El authMiddleware lo valida en cada request protegida y agrega req.user.

7.2 Roles

Rol	Descripción
root	Superior a todos. Acceso exclusivo al módulo ROOT. Usuario: ROOT, id=30.
admin	Administrador general
gerencia	Acceso a reportes y configuración estratégica
jefe_produccion	Supervisa el flujo de producción
asesor_comercial	Gestiona prospectos y ODPs comerciales
produccion	Operarios de producción
auxiliar_produccion	Auxiliar de producción
instalador	Ejecuta instalaciones en campo
conductor	Exclusivo del módulo de rutas de instalación
contabilidad	Facturación y caja
compras	Gestiona ODCs, SAPs y salidas de almacén

CRITICO: La tabla usuarios tiene un CHECK CONSTRAINT (usuarios_rol_check) que lista los roles explícitamente. Al agregar un rol nuevo hay que: (1) ALTER TYPE enum_usuarios_rol ADD VALUE; (2) DROP + recrear el CHECK CONSTRAINT.

7.3 Campo Especial: puede_gestionar_pv

Booleano en el modelo Usuario. Controla acceso al tab "Por Gestionar" en Pedidos PV. Debe estar definido en el modelo Sequelize o toJSON() no lo incluirá en el login.

8. Arquitectura Frontend

8.1 Estructura de Features

Cada módulo en frontend-web/src/features/<nombre>/ sigue este patrón:

- Página principal (ej. ComprasPage.tsx)
- components/ con modales y sub-componentes
- Slice Redux si maneja estado global

8.2 Módulos y Rutas

Feature	Ruta	Descripción
auth	/login	Login con JWT
odp	/odp	CRUD de ODPs + modal detalle completo
produccion	/produccion	Vista Kanban/tabla del estado de producción
instalaciones	/instalaciones	Vistas por rol: JefeView, InstaladorView, ConductorView
compras	/compras	ODC con tabs: SAPs, Órdenes, Perfilaría, Vidrios
contabilidad	/contabilidad	Facturación y caja
clientes	/clientes	CRUD clientes
prospectos	/prospectos	Pipeline CRM
toma-medidas	/toma-medidas	Gestión de mediciones
inventario	/inventario	Inventario de perfilaría
evidencias	/evidencias	Galería de fotos de instalación
usuarios	/usuarios	Administración de usuarios
pedidos-pv	/pedidos-pv	Gestión de pedidos de vidrios templados PV
facturas-salidas	/facturas-salidas	Control interno Facturas vs SA-XXXX
reportes	/reportes	Reportes y exportación
configuracion	/configuracion	Configuración global del sistema
root	/root	Panel de control total — solo rol root

8.3 Componentes Clave del Módulo ODP

Componente	Función
ODPDetailModal.tsx	Modal maestro con todos los tabs de una ODP
ODPForm.tsx	Formulario de creación/edición de ODP
TMModal.tsx	Toma de Medidas (dos momentos: antes y después de ODP)
SAPModal.tsx	Modal de solicitud de perfilaría
COTModal.tsx	Modal de cotización

Componente	Función
PrintableOP.tsx	PDF de Orden de Producción (window.print())
PrintableSAP.tsx	PDF de SAP (window.print())
PrintableProduccion.tsx	PDF de hoja de producción (window.print())

8.4 Estado Global (Redux)

- Store en store/store.ts con slices por feature
- Notificaciones en tiempo real vía useSocketNotifications hook
- socket.ts configura la conexión Socket.io compartida
- HTTP vía Axios; no usar fetch directamente
- Tema MUI centralizado en theme/theme.ts

8.5 Componentes Printable

Los componentes Printable*.tsx se renderizan en un div oculto y se imprimen con window.print(). No usan llamadas al backend; generan PDF directamente en el navegador.

9. Módulo ROOT

Exclusivo del rol root (usuario ROOT, id=30). Accesible en /root y /api/root/*. En el sidebar solo ve la sección "Sistema".

9.1 Tabs del Panel ROOT

Tab	Contenido
Resumen	Métricas generales del sistema
Base de Datos	Health check Supabase (SELECT 1 directo), tamaño de tablas
Almacenamiento	Health check Cloudinary (status.cloudinary.com/api/v2/status.json)
Servicios	Estado de servicios externos
Auditoría	Visor del log auditoria_log con filtros
Backup	Generación y descarga de backups
Mantenimiento	Operaciones de mantenimiento de BD
Alertas	Configuración de umbrales (alertas_umbral)
Catálogo	CRUD del catálogo de productos (movido desde Configuración)

9.2 Health Checks

- Supabase: SELECT 1 directo a la BD (no HTTP). Más confiable que pings a endpoints protegidos.
- Cloudinary: GET a status.cloudinary.com/api/v2/status.json.

10. Aplicación Móvil

Directorio mobile-app/. Usa Expo Router (file-based routing en app/), no React Navigation. Orientada a instaladores en campo.

- Expo Go para desarrollo, APK para distribución
- Expo Router: rutas basadas en archivos dentro de app/
- Consume la misma API REST del backend vía REACT_APP_API_URL

11. Guía de Desarrollo Local

11.1 Levantar el Backend

```
cd backend-api
npm install
npm run dev          # hot-reload en puerto 3001
```

11.2 Levantar el Frontend

```
cd frontend-web
npm install
npm start            # dev server en http://localhost:3000
```

11.3 Levantar la App Móvil

```
cd mobile-app
expo start
expo start --android # emulador Android
```

11.4 Otros Comandos Útiles

Comando	Descripción
npm run build (backend)	Compila TypeScript a dist/
npm run start (backend)	Producción (node dist/server.js)
npm run lint / lint:fix	ESLint en el backend
npm run format	Prettier en el backend
npm run build (frontend)	Build producción (CI=false)
node replace_urls.js	Reemplaza URLs hardcodeadas al cambiar entorno

11.5 Scripts One-Off (No ejecutar en producción)

Los siguientes archivos en backend-api/src/ son migraciones puntuales ya ejecutadas. No forman parte del flujo normal:

- seed.ts, seed_odps.ts
- migrate_nc.ts, db_master_fix.ts
- fix_name.ts, insertar_asesores.ts
- importar_clientes_excel.ts

12. Despliegue a Producción

12.1 Backend (Docker)

El backend usa un Dockerfile multi-stage basado en node:20-alpine:

```
# Build
docker build -t templex-backend .
# Run
docker run -p 3001:3001 --env-file .env templex-backend
```

12.2 Frontend

- Push a main dispara el build automático.
- Variable CI=false para no fallar por warnings de React.
- El backend debe tener FRONTEND_URL apuntando al dominio de producción.

12.3 CORS

- HTTP: whitelist + patrones *.netlify.app, *.vercel.app
- WebSocket: solo localhost:3000, FRONTEND_URL, *.netlify.app, *.vercel.app

12.4 Nginx (Reverse Proxy)

```
location /api/ {
    proxy_pass http://backend:3001;
}
location / {
    try_files $uri /index.html; # SPA routing
}
```

13. Notas Técnicas Adicionales

13.1 Importación de Modelos

SIEMPRE importar modelos desde `models/index.ts` para que las asociaciones estén cargadas. Dos modelos se importan directamente en sus controladores por convención:

- `Produccion` → `produccion.controller.ts`
- `ProgramacionInstalacion` → `instalacion.controller.ts`

13.2 ODP sin Timestamps

`ODP.timestamps = false`. La tabla `odp` no tiene `createdAt/updatedAt`. Usa el campo `fecha_creacion` gestionado manualmente.

13.3 Revertir Cambios (Auditoría)

El hook `beforeUpdate` guarda `instance.previous()` en `instance._auditAntes`. Si `datos_anteriores` es `null` en un `UPDATE`, el `revert` no puede ejecutarse porque no se capturó el estado previo.

13.4 Cron del PedidoPV

Existe un cron diario a las 8am que revisa `PedidoPVs` con tardanza y genera alertas.

13.5 No Hay Tests Automatizados

No existen tests unitarios ni de integración en backend ni frontend. Toda validación es manual.

13.6 Módulo de Cotizaciones

IMPORTANTE: "cotizaciones" en el contexto del equipo normalmente se refiere al `COTModal` dentro del módulo `ODP`, NO al módulo `/cotizaciones` que está en construcción y solo visible para admin.

— Fin del documento —